Tool-Augmented Question-Centric Retrieval Using QuIM-RAG for Reliable Knowledge-Grounded Question Answering

Pradeep Kumar Galla
Student,Department of CSE
R.V.R. & J.C. College of Engineering
Guntur, Andhra Pradesh, India
pradeepkumargalla57@gmail.com

Naga Avinash Babu Enadula
Student, Department of CSE
R.V.R. & J.C. College of Engineering
Guntur, Andhra Pradesh, India
enadulaavinash@gmail.com

Dr. Sreelatha Malempati
Professor, Department of CSE
R.V.R. & J.C. College of Engineering
Guntur, Andhra Pradesh, India
lathamoturicse@gmail.com

***Abstract*—Retrieval-augmented generation (RAG) is an effective means of increasing the factual reliability of large language models (LLMs) when utilizing external knowledge to ground generated responses. Current implementations of RAG use text-only corpora to retrieve information using a chunk-based retrieval technique which often causes semantic misalignment between the chunk retrieved and the intended semantic meaning, information dilution because of the volume of chunks returned from the corpus, and hallucination of facts in domain real-world scenarios. To overcome these limitations, QuIM-RAG introduced a novel approach to question-based retrieval that utilizes a prototype-driven inverted index to increase retrieval accuracy and grounded responses compared to previous RAG implementations. However, QuIM-RAG makes assumptions related to the cleanliness and preprocessing of the text corpus and does not address the challenges of heterogeneous web-based data and the practical implementation of retrieval systems.**

In this work,we extend the QuIM-RAG framework through data-centric and system-level enhancements designed for realistic web-based knowledge sources. Our approach incorporates tool-augmented corpus construction from heterogeneous data, including textual content, structured HTML tables, embedded images via optical character recognition (OCR), and linked PDF documents. The extracted information is normalized into a unified textual representation and integrated into a question centric indexing pipeline using prototype-based inverted indexing. Additionally,we incorporate auxiliary components such as query rewriting and cross-encoder reranking to improve robustness under ambiguous queries,while clearly distinguishing these from core retrieval mechanisms.

We evaluate the proposed system on a medium-scale, domain specific institutional corpus using BERTScore and RAG-oriented metrics, including faithfulness, answer relevance, context precision, and harmfulness, and compare it against lexical (BM25),dense, and hybrid retrieval baselines. Experimental results demonstrate that the proposed extensions achieve competitive semantic accuracy while improving grounding and reliability in domain-specific question answering. These findings highlight the importance of data-centric and system-level enhancements for enabling practical deployment of question-centric RAG frameworks in real-world environments.

***Index Terms*—Retrieval-Augmented Generation, Question-Centric Retrieval, QuIM-RAG, Web Knowledge Extraction, Domain-Specific Question Answering, Large Language Models**

I. Introduction

Large Language Models have made a lot of progress in understanding and generating natural language. This has led to the development of applications, including chatbots and specialized knowledge systems. However these models still have limitations. They rely heavily on the knowledge they gained during their training phase. The problem is that this knowledge can become outdated or incomplete. As a result Large Language Models can sometimes produce inaccurate information. This inaccuracy restricts their usefulness in real-world applications. They can make things up which is an issue. Researchers have noted this problem citing it as a challenge [5], [6]. Large Language Models need to be improved to become more reliable. They have to be able to provide information. This is crucial for their use, in applications.

Retrieval-Augmented Generation or RAG is a strategy that helps reduce hallucination in LLMs by adding knowledge retrieval. This method combines searching through document collections with generating text. RAG allows models to create responses based on retrieved information, than just what they already know. Since RAG was introduced many RAG-based systems have been developed to improve the retrieval process [1]. These systems have looked into retrieval methods combining different indexing techniques and improving the way results are ranked to make retrieval more accurate and reliable. RAG systems aim to provide factual and accurate responses.

However, chunk-centric retrieval is used by the majority of current RAG systems, in which user queries are directly compared to fixed-length page fragments in an embedding space. There are two main difficulties with this strategy. First, inappropriate retrievals may arise from a semantic mismatch between user queries and content chunks, especially for implicitly worded or underspecified searches. Second, chunk-centric retrieval is vulnerable to information dilution as corpus size grows, recovering several loosely connected passages that worsen downstream generation and raise the chance of hallucinations [13].

In order to overcome these drawbacks, QuIM-RAG (Question-to-Question Inverted Matching) developed a question-centric retrieval paradigm that replaces document chunks with automatically created questions [2]. This method generates representative questions for each chunk and matches user queries in a shared embedding space with these questions. In order to facilitate effective coarse-to-fine retrieval, QuIM-RAG also uses a prototype-based inverted index, grouping semantically related queries into prototype vectors. Compared to traditional RAG systems, this architecture decreases information dilution and increases retrieval precision.

While QuIM-RAG gives an understanding of how to make retrieval more reliable it mainly works with clean and prepared text data. It does not directly deal with the problems that come with real-world web data. In real-life situations in big organizations and companies useful information is often found in different types of sources. These sources include: (i) tables, (ii) images, (iii) scanned papers and (iv) linked PDF files. This makes it hard to get the information we need. QuIM-RAG and similar methods need to be adjusted to handle these challenges. Conventional RAG and QuIM-RAG pipelines are not designed to directly ingest or effectively utilize such semi-structured and non-textual knowledge without substantial preprocessing.

In this work, we extend the QuIM-RAG framework toward real-world applicability through data-centric and system-level enhancements. We introduce a tool-augmented web corpus construction pipeline that integrates automated extraction of domain knowledge from diverse web artifacts, including textual content, structured tables, images via optical character recognition (OCR), and PDF documents. The extracted knowledge is normalized into a unified textual representation and incorporated into a question-centric indexing pipeline using prototype-based inverted indexing. Additionally, we incorporate auxiliary components such as query rewriting and cross-encoder reranking to improve retrieval robustness and precision under ambiguous queries.

We evaluate the proposed system on a medium-scale, domain-specific corpus using BERTScore and RAG-oriented metrics, including faithfulness, answer relevance, and context precision, and compare performance against lexical, dense, and hybrid retrieval baselines. Experimental results demonstrate that the proposed extensions achieve competitive semantic accuracy while improving grounding and reliability in domain-specific question answering. These results emphasize how crucial data-centric and system-level improvements are to the realistic implementation of question-centric RAG frameworks in practical settings.

II. Related Work

This section summarizes previous studies on web-based knowledge acquisition, question-centric retrieval, retrieval-augmented generation, and assessment of hallucinations in big language models. We place our work in the context of this literature and explain how it builds upon previous methods.

A. Retrieval-Augmented Generation

The Retrieval-Augmented Generation system or RAG for short brings together language modeling and knowledge retrieval to make sure the facts are right for tasks that need a lot of knowledge. The basic RAG framework that Lewis and others came up with combines finding documents with generating text, one sequence at a time. This means that big language models can give answers based on the documents they find of just using what they already know. This way of doing things works well for answering questions and checking facts and it helps stop language models from making things up [1]. This approach demonstrated strong performance on open-domain question answering and fact verification benchmarks, establishing RAG as a core paradigm for mitigating hallucinations in LLM outputs.

People have done research to make the part of RAG that finds documents work better. They have tried using ways to search, like looking at the meaning of words and their context and they have made the document encoders better. They have also tried rearranging the results to get the relevant ones first. Even with these improvements most RAG systems still treat each document as a single unit when they search. This does not work well when the documents are big or very different because the search results are not always, about what the user is looking for [13]. As a result the search results often include things that're not really relevant which can make the answers worse. The RAG system is still a way to get accurate facts but it needs to be improved to work better with big or heterogeneous corpora like the Retrieval-Augmented Generation system.

B. Question-Centric Retrieval and QuIM-RAG

Recent research has investigated question-centric retrieval paradigms, in which retrieval units are created from questions rather than raw document segments, in order to overcome the drawbacks of chunk-centric retrieval. Question-to-Question Inverted Matching was established by QuIM-RAG, which creates representative questions for every document chunk and performs retrieval by comparing user queries with these questions in a shared embedding space [2]. This approach improves semantic alignment between queries and indexed knowledge.

In order to facilitate effective coarse-to-fine retrieval, QuIM-RAG also uses a prototype-based inverted index, grouping semantically related queries into prototype vectors. Compared to traditional dense RAG pipelines, this architecture minimizes information dilution and decreases retrieval ambiguity. However, structured tables, visual information, and scanned documents are examples of real-world online data issues that are not addressed by the original QuIM-RAG formulation, which mostly assumes clean, text-only corpora.

C. Web-Based Knowledge Extraction and Domain Corpora

When we are dealing with life situations information about a specific area of knowledge is often found in different types of things we see on the web. These things include tables on web

pages, pictures and files like PDFs [11], [10]. Most of the time the systems that help us find answers to our questions called RAG pipelines do not use these sources of information. They just look at them as regular text. This means they do not get all the information they could and it takes them longer to find the answers.

Other people have studied how to get information from the web. They have looked at ways to understand tables get text from pictures and break down documents. However these methods are not often used together with RAG systems.

New studies show that using methods that focus on the data is important to make the models that answer our questions reliable. This is especially true for enterprise and institutions where the information we need's not just in the text we read [21]. The systems we use now to answer questions usually rely on information that has already been prepared and they do not have a good way to automatically get information from all the different sources on the web. Our research is trying to fix this problem by using tools to get information, from the web and adding it to the system that answers our questions.

D. Hallucination and Evaluation of RAG Systems

Hallucination is still a problem in systems that use large language models especially when the information they retrieve is not good or is not related to the task. Researchers have looked into this issue. Suggested ways to make it better such as getting better information checking for consistency and evaluating how accurate the information is [14].

Some studies have also come up with ways to measure how well these systems work, especially for systems that use retrieval to help generate answers. For example BERTScore checks how similar the answers given by the system are to the documents. There are also frameworks like RAGAS that use metrics, including how faithful the answers are, how relevant they are and how precise the context is. Our approach to evaluating these systems makes use of different metrics and tests them in various ways. We use benchmarks that focus on the words used, retrieval, a mix of both and questions to get a better picture of how well these systems work. This helps us understand what works and what does not and how we can make these systems better. We are building on the work of others in this area including research on hallucination [5], [6], [14] BERTScore [19] and RAGAS [4]. Our goal is to make these systems more accurate and reliable by improving how they retrieve and generate information. Hallucination and retrieval are key, to achieving this goal. By focusing on hallucination and retrieval we can make these systems more trustworthy.

E. Positioning of Our Work

Unlike previous work, our method expands QuIM-RAG by making system-level and data-centric improvements targeted at real-world implementation. Instead of suggesting a novel retrieval paradigm, we concentrate on making it possible for question-centric retrieval to function well across diverse web-based information sources. Our work improves the applicability of question-centric RAG frameworks beyond controlled benchmark datasets and into realistic deployment situations by combining tool-augmented corpus building, prototype-based indexing within a unified pipeline, and thorough RAG-oriented evaluation.

III. System Overview

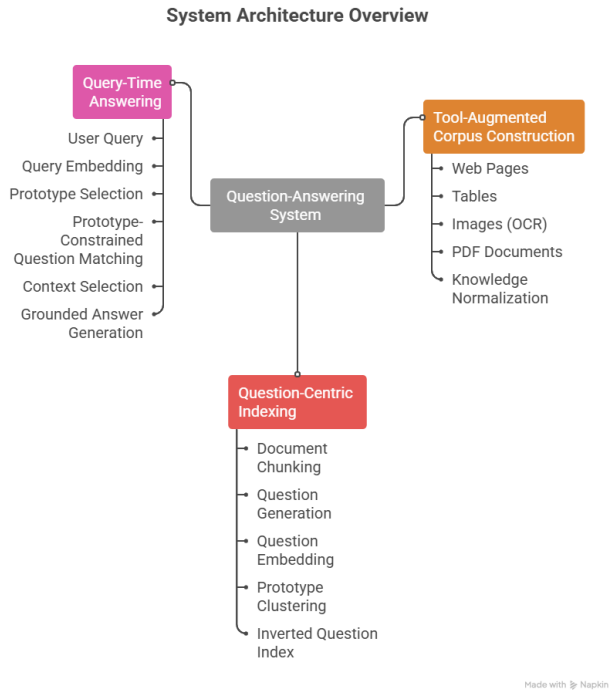

Fig. 1. Overview of the proposed tool-augmented QuIM-RAG system.

The new system makes the QuIM-RAG question-focused retrieval framework better with improvements that are based on data and apply to the system so it can be used in real life. As shown in Fig. 1 the system is divided into three parts:(i) tool-augmented web corpus construction, (ii) question-centric knowledge indexing, and (iii) query-time retrieval and answer generation. The QuIM-RAG question-focused retrieval framework is set up in a way that allows it to work well with kinds of real-world data and it still follows the basic ideas of question-focused retrieval. The QuIM-RAG question-focused retrieval framework is made to be flexible.

A. Tool-Augmented Web Corpus Construction

The initial phase builds a knowledge base that's specific to a domain. It uses web resources. Unlike systems that rely on pre-processed text data the suggested system directly uses web content. It uses methods to gather a variety of information.

he pipeline starts by crawling the web. It only looks at websites within a domain. Then it extracts text, tables, images and PDF files. Textual content is refined by eliminating standard elements like headers, navigation bars, and footers.

Tables are converted into text. They still show which information is in each row and column. This makes it easy to use

them later. Images are analyzed to get the text out of them. The system uses a OCR tool to recognize text in images and . It makes the text recognition better by reducing noise and adjusting the image. PDF files are also analyzed. The system extracts the text, from them.

All the gathered data is put into one text format. This makes a combined collection that can be indexed. The system can work well with types of web-based knowledge. It does not just work with text data. The system uses web resources directly. It gathers information from the web.

B. Question-Centric Knowledge Indexing

In the phase the corpus that was developed is looked at closely using the question-focused indexing approach that is used in QuIM-RAG. The documents are broken down into parts that overlap so that the meaning of the text stays the same even when the parts are not too long. A language model that has been adjusted to follow instructions creates questions that represent each part and show what information it contains.

These questions that were created are the things that are used to find information instead of using the parts of the documents directly. Each question is put into a space where meanings are shown in a consistent way so that questions from documents and questions from users are represented in the same way. To make it easier to find information and to avoid getting much extra information the questions are grouped together using a method called centroid-based clustering. The middle points of these groups, which are called prototypes are, like representatives of groups of questions.

An inverted index maps each prototype to its associated questions and corresponding document chunks, enabling efficient narrowing of the search space during query-time retrieval.

C. Query-Time Retrieval and Answer Generation

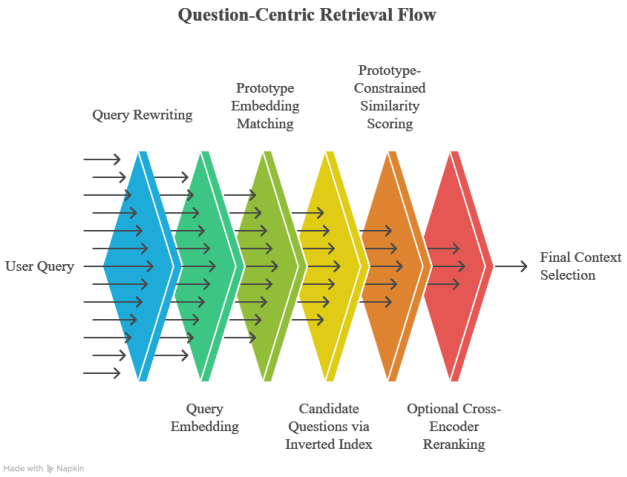

Fig. 2. Question-centric retrieval and prototype-constrained matching process.

During inference, a user query is optionally reformulated using a query rewriting module to improve clarity and retrieval effectiveness. The query is then embedded into the same semantic space as the indexed questions. The query embedding is compared against prototype embeddings, and the nearest prototype is selected to identify a relevant subset of candidate questions.

Within this reduced candidate set, dense similarity matching is performed to retrieve the most relevant questions and their associated document chunks. An optional cross-encoder reranking step further refines ranking precision. The top-ranked chunks are then provided as context to a language model, which generates responses grounded in retrieved evidence.

If no adequately pertinent context is found, the system refrains from responding, thus minimizing unsupported or fabricated outputs. This regulated generation process enhances dependability in domain-specific question-answering situations

D. Design Rationale

The system architecture is structured to divide data acquisition, indexing, and retrieval into separate modules. Tool-enhanced corpus creation tackles the constraints of solely text-based ingestion, whereas question-focused indexing enhances semantic coherence and minimizes information loss. Prototype-based indexing facilitates effective coarse-to-fine retrieval, while additional elements like query rewriting and reranking improve robustness without altering the fundamental retrieval framework. Collectively, these design decisions enhance QuIM-RAG for effective use in actual settings

IV. Methodology

The suggested methodology is explained in this part, including how grounded answer creation, prototype-based retrieval, question-centric indexing, and tool-augmented corpus development are all combined into a single system. With data-centric and system-level improvements intended for real-world online data and domain-specific deployments, the methodology expands the QuIM-RAG framework.

A. Tool-Augmented Web Corpus Construction

The initial phase emphasizes creating a specialized knowledge corpus from actual online sources. In contrast to traditional RAG pipelines that rely on pre-processed text datasets, the suggested method automates the collection of data and knowledge extraction from diverse web materials.

Given a target domain, a crawler retrieves web pages under domain and depth constraints. For each page, multiple extraction techniques are applied: visible textual content is extracted after removing boilerplate elements, structured HTML tables are parsed and converted into textual representations preserving row–column semantics, embedded Image-based textual information is extracted using optical character recognition (OCR) with the Tesseract engine. To improve recognition accuracy under real-world conditions, preprocessing steps including grayscale conversion, noise reduction, and thresholding are applied prior to OCR. The extracted text is further cleaned and normalized before integration into the unified corpus.

TABLE I
WEB KNOWLEDGE EXTRACTION MODULES

Content Type	Extraction Mechanism
Text	HTML parsing and boilerplate removal
Tables	Structured HTML table parsing
Images	OCR-based text extraction
PDFs	Document text parsing

B. Knowledge Chunking and Question Generation

To preserve semantic coherence while adhering to the context constraints of language models, the normalized corpus is divided into overlapping sections. Let a document D be divided into chunks $\{c_1, c_2, \ldots, c_n\}$, with a fixed number of tokens with controlled overlap in each chunk.

An instruction-tuned language model creates a set of representative questions $_{i1}, q_{i2}, \ldots, q_{ik}\}$ for every chunk c_i so that each question can be answered using only the content of c_i. Passive knowledge is converted into a query-aligned representation space through this approach.

Chunks that do not yield valid questions are excluded from indexing, ensuring that each indexed unit is associated with at least one explicit question.

C. Question Embedding and Prototype-Based Index Construction

A sentence embedding model is used to embed each generated question q into a shared semantic vector space:

$$e_q = f_{\text{embed}}(q), \quad e_q \in \mathbb{R}^d$$

where $f_{\text{embed}}(\cdot)$ denotes the embedding function.

Centroid-based clustering is used to cluster question embeddings in order to increase retrieval efficiency and decrease information dilution. Let the set of all questions be represented by Q. Clustering yields K prototype vectors $\{p_1, p_2, \ldots, p_K\}$, each of which has the following definition:

$$p_k = \frac{1}{|C_k|} \sum_{q \in C_k} e_q$$

where C_k denotes the set of questions assigned to prototype p_k.

An inverted index is constructed that maps each prototype to its associated questions and corresponding document chunks, enabling efficient narrowing of the search space during query-time retrieval.

D. Query-Time Question-Centric Retrieval

A query rewriting module is used to potentially reformulate a user query u during inference time in order to enhance clarity. The same semantic space contains the embedded query:

$$e_u = f_{\text{embed}}(u)$$

The closest prototype p^* is chosen using cosine similarity after the query embedding is compared to prototype embeddings:

$$p^* = \arg\max_{p_k} \ \text{cosine}(e_u, p_k)$$

The inverted index is used to retrieve candidate questions related to p^*. Dense similarity matching between the query embedding and candidate question embeddings is carried out within this limited collection. Ranking accuracy is further improved by an optional cross-encoder reranking phase. The matching document chunks utilized as retrieval context are determined by the top-ranked questions.

E. Grounded Answer Generation and Refusal Mechanism

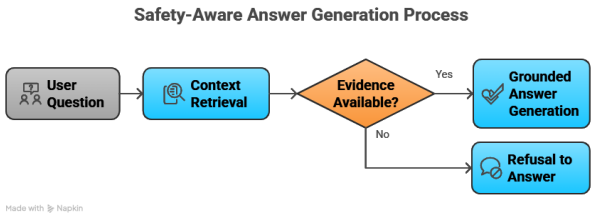

Fig. 3. Grounded answer generation with refusal mechanism. The system generates responses only when sufficient evidence is retrieved, otherwise abstains from answering.

The system makes sure that it only gives answers when it has information to back them up like you can see in Fig. 3.

A language model gets the information that was chosen and uses it to come up with answers. The language model only uses the information that it found to make these answers.

The system will not give an answer if it does not have good information. The system says no. Gives a message saying that it cannot answer the question if it does not have enough information.

This way of doing things helps to make sure that the answers the system gives are based on information that it found. The system reduces the number of times it makes things up and makes sure that its answers are based on what it knows, which is called grounding. The system ensures that language model replies are backed by retrieved knowledge this approach reduces hallucinations. Ensures tight grounding of language model replies.

F. Methodology Summary

The proposed methodology integrates tool-augmented corpus construction, question-centric indexing, prototype-based inverted retrieval, and grounded answer generation into a unified framework. By extending QuIM-RAG with data-centric and system-level enhancements, the approach enables effective operation over heterogeneous web-based knowledge sources.

Unlike conventional RAG pipelines that rely on clean textual corpora, the proposed system incorporates automated extraction from diverse data modalities, including structured tables, images, and PDF documents, thereby improving knowledge coverage in real-world settings. The adoption of question-centric indexing preserves semantic alignment between queries

and indexed knowledge, while prototype-based clustering enables efficient coarse-to-fine retrieval.

At query time, retrieval is constrained to semantically relevant regions of the question space, followed by similarity matching and optional reranking to improve precision. The integration of a refusal mechanism further enforces strict grounding by preventing unsupported answer generation.

Overall, the methodology demonstrates how system-level design and data-centric enhancements can extend existing question-centric RAG frameworks toward reliable and practical domain-specific deployment.

TABLE II
COMPARISON OF RAG, QUIM-RAG, AND PROPOSED SYSTEM

Feature	RAG	QuIM-RAG	Proposed
Retrieval Unit	Chunks	Chunks	Chunks & Questions
Indexing	Dense/Hybrid	Prototype	Prototype
Data Type	Text	Text	Multi-modal
Corpus	Preprocessed	Preprocessed	Tool-Augmented + Preprocessed
Retrieval	Global	Prototype	Refined
Grounding	Limited	Moderate	Refusal-based
Deployment	Low	Medium	High

V. EXPERIMENTAL SETUP

This section describes the experimental configuration used to evaluate the proposed tool-augmented QuIM-RAG framework. We detail the dataset construction, model components, indexing strategy, retrieval pipeline, baseline systems, and reproducibility settings to enable independent replication.

A. Dataset and Corpus Construction

We made our experimental corpus using the tool-enhanced web extraction pipeline that we talked about earlier. The data comes from institutional websites and has many different types of things like normal text, tables from websites text from pictures that we got using special software and PDF files that go with them.

We made all the data look the same so it is easy to work with. We got rid of things like headers and footers that do not really matter. Then we broke up the documents into pieces so that they make sense and are not too long for the language models we use later.

TABLE III
CORPUS STATISTICS

Component	Quantity
Web pages crawled	∼40
Normalized documents	∼40
Knowledge chunks	∼108
Generated questions	∼615
Prototype clusters	256
Avg. questions per chunk	∼5–6

The resulting dataset represents a medium-scale, domain-specific corpus that reflects realistic institutional deployment scenarios. While limited in size, it is sufficient to evaluate system behavior and validate design choices in practical settings.

B. Question Generation Configuration

We look at every part of the knowledge chunk to create questions using a instruction-tuned language model. When we make these questions we make sure they meet some rules. The questions we create have to be answerable using the related part of the knowledge. Prompt design ensures that generated questions satisfy the following constraints:

- Each question is answerable solely using the corresponding chunk
- External knowledge or inference beyond the chunk is avoided
- Redundant, vague, or speculative questions are filtered

We only keep the parts of the knowledge that give us least one good question. This helps us make sure that every part of the knowledge is connected to questions that can be answered. It also helps us find the answers.

C. Embedding Model and Vector Representation

All generated questions, document chunks, and user queries are embedded into a shared semantic vector space using a sentence-level transformer model.

- Embedding model: `BAAI/bge-large-en-v1.5`
- Embedding dimensionality: 1024
- Vector normalization: L2 normalization
- Similarity function: Cosine similarity

Using the same model helps us to maintain consistency when we are indexing and retrieving information.

D. Prototype-Based Index Construction

The embeddings of the generated questions are represented by e_{q_i} and the collection of generated questions is represented by $Q = "q_1, q_2, \ldots, q_n\}$. We use centroid-based clustering to group these embeddings. This helps to minimize information dilution and make the retrieval process simpler.

The number of clusters, which we call prototypes is determined by the size of Q. Let us say we have a number of prototypes. Every prototype is described in a way.

$$p_k = \frac{1}{|C_k|} \sum_{q \in C_k} e_q$$

For each prototype we have a set of questions. We denote the set of questions for prototype p_k as C_k.

An inverted index maps each prototype to its associated questions and corresponding document chunks, enabling efficient narrowing of the retrieval space during query-time inference.

E. Vector Database Backend

We use a vector database to store all the embeddings and other important information. This makes it easier to find the matches to something.

- Vector database: ChromaDB
- ANN method: HNSW-based indexing
- Stored metadata: question text, chunk identifier, source URL

When new data is ingested, prototype clustering and index generation are carried out offline and updated gradually.

F. Query-Time Retrieval Components

This part talks about the query-time retrieval pipeline. It is the way the suggested system works when it is actually being used.

1) Query Rewriting: User queries often exhibit ambiguity or underspecification. To improve retrieval robustness, a lightweight query rewriting module is optionally applied to reformulate the query into a clearer and self-contained form without altering its semantic intent. The rewritten query is used for retrieval, while the original query is preserved for answer generation. This component is treated as an auxiliary enhancement rather than a core methodological contribution.

2) Query Embedding: The (rewritten) query u is embedded using the same embedding model employed during indexing:

$$e_u = f_{\text{embed}}(u), \quad e_u \in \mathbb{R}^{1024}$$

Embedding normalization ensures consistent cosine similarity computation across all retrieval stages.

3) Prototype Selection via Query–Prototype Matching: We look at the embedding of the question, which's e_u and compare it to each of the prototype embeddings, which are $\{p_1, p_2, \ldots, p_K\}$. We do this to find the general meaning of the query. We pick the prototype that's closest to the query u.

$$p^* = \arg\max_{p_k} \cos(e_u, p_k)$$

This step helps us find the part of the system that is most relevant, to the query u so we can look at a smaller group of things that might be what we are looking for instead of looking at everything.

4) Candidate Question Retrieval: We use the prototype p^* to get a list of possible questions Q_{p^*} from the inverted index. Each of these questions is connected to some parts of documents. This helps us filter out questions that do not make sense. We only look at questions that are related.

5) Dense Similarity Matching: Next we compare the query to each question in Q_{p^*} to see how similar they are. We do this by using the cosine of the angle, between the query embedding e_u. The question embedding e_q:

$$\text{score}(q) = \cos(e_u, e_q)$$

We then rank the questions based on these similarity scores. This helps us find the matching questions.

6) Cross-Encoder Reranking: The top-k candidates go through a encoder reranker to make the ranking more precise.

- Reranker model: `cross-encoder/ms-marc-MiniLM-L-6-v2`

The cross-encoder jointly encodes the query and candidate question, capturing deeper semantic interactions beyond embedding-based similarity.

This reranker looks at the query. Each candidate question together. It does not just check if they are similar. It tries to understand them. It helps to pick candidate questions. The cross-encoder reranker is used. The top-k candidates are reranked.

7) Context Selection: We use the matching parts of documents as the context. These parts are chosen based on the questions that're most likely to be relevant. We make sure that we do not use the part of a document more than once and that the context is not too long. This helps the model that generates answers to work properly.

G. Answer Generation and Refusal Mechanism

We use a kind of language model to generate answers. This model is given the context we found and uses it to create a response. The model only uses the given context. Does not make things up. This helps to ensure that the answers are accurate.

If we cannot find a context for a question the system will not give an answer. Instead it will say that it cannot answer the question. This makes the system more reliable especially when it comes to answering questions, about topics. The answer generation model and the refusal mechanism work together to prevent the system from giving answers that are not based on the context. The answer generation model and the refusal mechanism are parts of the system.

H. Baseline Systems

The proposed method is compared against the following baselines:

TABLE IV
BASELINE METHODS

Method	Description
BM25	Lexical retrieval using TF-IDF scoring
Dense	Chunk-centric dense retrieval
Hybrid	Combined lexical and dense retrieval
QuIM-RAG	Question-centric prototype-based retrieval
Proposed	Extended QuIM-RAG (tool-augmented system)

All methods use the same models for generation and embedding and the same prompts to ensure a fair comparison. The suggested method, called Extended QuIM-RAG is compared to these baselines. QuIM-RAG and Extended QuIM-RAG are question- prototype-based retrieval methods. BM25, Dense and Hybrid are baseline methods used for comparison, with QuIM-RAG and Extended QuIM-RAG.

I. Hardware and Runtime Environment

Experiments were conducted using a cloud-based environment with GPU acceleration.

- Platform: Kaggle Notebooks
- GPU: $\sim$16 GB GPU memory
- CPU: Multi-core processor
- RAM: $\geq$32 GB

This setup is, like a useful and accessible computational environment, not a high-performance one.

J. Reproducibility Protocol

We made sure to use the clustering parameters control random seeds and use the same prompt templates every time. Our pipeline is designed in a way, which helps us ensure reproducibility at every step from developing the corpus to generating answers. We followed this protocol for corpus development, indexing, retrieval and answer generation.

VI. Evaluation Metrics

We need to evaluate Retrieval-Augmented Generation systems using metrics that look at how they are grounded how relevant they are, how safe they are and how semantically correct they are. The old way of measuring things like looking at how many words overlap is not good enough because it does not check if the facts are consistent and if the system is using the evidence it retrieved correctly. So we use a way of evaluating that includes metrics that care about safety, retrieval, and semantics.

A. Semantic Answer Quality (BERTScore)

To see how similar the answers are, we use something called BERTScore. It looks at how similar the words are, in a special space that understands the context.

We use BERTScore to calculate how good the generated answers are compared to the reference answers. It does this by looking at how similar the words are using something called cosine similarity. Then it calculates three things: precision P, recall R and F1 score F_1. It does this by comparing the generated answer A_g with the reference answer A_r.

$$P = \frac{1}{|A_g|} \sum_{t_g \in A_g} \max_{t_r \in A_r} \cos(e_{t_g}, e_{t_r})$$

$$R = \frac{1}{|A_r|} \sum_{t_r \in A_r} \max_{t_g \in A_g} \cos(e_{t_r}, e_{t_g})$$

$$F_1 = \frac{2PR}{P + R}$$

BERTScore is well suited for generative QA tasks, as it captures semantic equivalence beyond exact token matching.

B. Faithfulness

We want to know if the answer we get is really true and based on what we retrieved . This is called faithfulness. It is, like seeing if the answer is supported by what we retrieved.

It is computed by:

- We break down the response into facts/claims
- We check each claim, against the context we retrieved
- Measuring the fraction of supported claims

If N_v is the number of claims that are verified and N is the number of claims:

$$\text{Faithfulness} = \frac{N_v}{N}$$

Higher values indicate stronger grounding and reduced hallucination.

C. Answer Relevance

We check to see if the answer we get is really what we were looking for. This is called answer relevance. It is like seeing if the response really answers the question we asked.

Let q denote the query and $\{\hat{q}_1, \hat{q}_2, \ldots, \hat{q}_m\}$ denote questions generated from the answer. Relevance is defined as:

$$AR = \frac{1}{m} \sum_{i=1}^{m} \cos(e_q, e_{\hat{q}_i})$$

This metric captures alignment between answer content and query intent.

D. Context Precision

We look at how much of the information we get back is really useful for answering the question. This is called context precision. It is like seeing how much of what we find is really what we need.

Let S denote all retrieved sentences and $S_r \subseteq S$ denote relevant sentences:

$$\text{Context Precision} = \frac{|S_r|}{|S|}$$

Higher values indicate more focused retrieval with less information dilution.

E. Harmfulness

We need to check the responses to see if they have any harmful things in them. We want to make sure they are safe. Let us say N is all the things that are said and H is all the things that are said

$$\text{Harmfulness} = \frac{H}{N}$$

Lower values indicate safer and more controlled generation.

F. Evaluation Strategy

Two evaluation strategies are employed:

1) **Baseline Comparison**: We look at how QuIM-RAG dense, hybrid and BM25 retrieval techniques work with a selection of queries.
2) **Full System Evaluation**: We use the suggested system to answer a lot of domain- questions and see how well it does with grounding, relevance and safety.

We do all the evaluations, in situations to see how the retrieval approach affects the results of the QuIM-RAG retrieval approach and the BM25 retrieval approach and the dense retrieval approach and the hybrid retrieval approach.

G. Summary of Metrics

This combination of metrics provides a comprehensive evaluation of both answer quality and reliability in RAG systems.

VII. Results and Discussion

The results of our tool-augmented QuIM-RAG framework are presented here. We also look at how it retrieves information the quality of its answers and its safety. We compare our system to baselines: lexical, dense, hybrid and QuIM-RAG

TABLE V
EVALUATION METRICS SUMMARY

Metric	Objective
BERTScore	Semantic answer correctness
Faithfulness	Grounding in retrieved context
Answer Relevance	Query–answer alignment
Context Precision	Retrieval efficiency
Harmfulness	Safety and appropriateness

A. Baseline Comparison Results

We start by testing a set of question-answer pairs to see how well each system retrieves information. The BERTScore precision, recall and F1 scores, for each approach are shown in Table VI. We use these scores to evaluate the retrieval performance of our QuIM-RAG framework and the baselines. The results help us understand how well our system works compared to the others. Our QuIM-RAG framework is tested to see if it can retrieve information effectively. The results of the tests are compared to those of the baselines

TABLE VI
BASELINE RETRIEVAL COMPARISON (BERTSCORE)

Method	Precision	Recall	F1
BM25	0.845	0.929	0.885
Dense	0.860	0.955	0.905
Hybrid	0.857	0.956	0.903
QuIM-RAG	0.853	0.951	0.899
Proposed	0.868	0.950	0.907

Dense and hybrid retrieval techniques are really good at finding a lot of relevant content like we can see in Table VI. The problem is that they often do not find the most relevant content because they also find a lot of irrelevant information. This happens because when answers are created they can be affected by background information that's not really related to the question.

The QuIM-RAG system uses question- indexing, which means it focuses on the question when it is searching for information. This helps the system find relevant information than other techniques that break up the text into small chunks. The suggested system also uses tools to help create a better collection of text and to refine the search results. This means that the system can find relevant and higher quality information, which is what we want from dense and hybrid retrieval techniques, like these.

The approach works well. Gives consistent results even with small improvements in BERTScore. It provides reliable answers that make sense in context as shown by better scores in metrics like faithfulness and context precision. This shows that we can combine data improvements with question-focused retrieval in real-life situations effectively. The method achieves performance and exhibits consistent retrieval behavior. It yields contextually supported responses as seen by gains, in grounding-oriented metrics. The approach works well in real-world deployment circumstances.

VIII. SCALABILITY CONSIDERATIONS

The system we are talking about has some scalability features that we will discuss here. We also want to understand how the system works when we have a lot of questions and a big corpus of data. We will look at how this affects the speed of retrieval.

A. Limitations of Chunk-Centric Retrieval

When we use chunk-centric Retrieval-Augmented Generation systems we have to compare a query to every single document chunk in the corpus. Let us say we have a number of dimensions for our embeddings, which we will call d and a certain number of chunks, which we will call N. The thing is, the more chunks we have the more memory we need and the longer it takes to retrieve the information even if we use special techniques to speed it up like approximate closest neighbor techniques and this is a problem, for scalability of the Retrieval-Augmented Generation systems and the chunk-centric retrieval method.

As corpus size increases, two challenges become prominent:

- **Retrieval Cost Growth:** The number of chunks goes up which makes it take longer to retrieve information. This means we need to use search algorithms. When the corpus is larger it's also more likely to retrieve not very relevant information, which can hurt answer quality.
- **Information Dilution:** Larger corpora increase the likelihood of retrieving partially relevant or noisy contexts, which can negatively impact answer generation.

To deal with these problems we need retrieval algorithms that can narrow down the search space without losing relevance.

B. Effect of Question-Centric Indexing and Prototype-Based Retrieval

The suggested solution uses question- indexing and prototype-based clustering like, in QuIM-RAG to tackle these issues. By grouping related questions into prototype groups we limit retrieval to a part of the search area. This approach helps to reduce retrieval latency and improve answer creation. The prototype-based clustering helps to minimize partially relevant contexts. The question-centric indexing enables the retrieval of relevant information.

In order to reduce the number of comparisons the system first figures out which prototype is most relevant at query time. Then it does similarity matching within that subset. This approach, which goes from coarse to fine makes the process more efficient while keeping the meaning aligned.

The retrieval pipelines design shows it can handle data than global chunk-based retrieval methods. This is even with a sized dataset. We will check how it works with datasets, in future studies.

C. Impact on Retrieval Latency and Memory Usage

The method we suggest reduces the number of comparisons needed for inference. It does this by looking at a subset of questions that are semantically similar. Compared to chunk-based retrieval doing dense similarity matching on a smaller set of embeddings can make retrieval faster. This is because there are embeddings to process.

The system uses embeddings for every query, which is a good thing. This means it can use vector database operations often. This could make the system work faster and use resources even when the corpus gets really big.

D. Scalability of Tool-Augmented Corpus Construction

The system gets data with the help of tools. It does this before it starts working. It uses tools like OCR and PDF processing to get the data from documents. The system does all of this one time for each document so it does not slow down the system when someone is searching for something.

The system uses a way of indexing that is centered around questions and it groups similar things together. This helps to keep the data organized even when the system is getting data from different sources. This makes the system really good, at handling all kinds of knowledge without getting too complicated when someone is searching for something.

E. Incremental Index Updates

Our approach supports updates, which are common, in real-world settings. When new documents are added:

1) New chunks are generated
2) Questions are produced and embedded
3) Embeddings are assigned to the nearest existing prototype
4) The inverted index is updated accordingly

This approach avoids re-indexing the corpus. Prototype recomputation may be done sometimes. Not every time an update is made.

F. Scalability Trade-offs

Prototype-based retrieval introduces two offline computations: question creation and grouping. However these costs are made up for during indexing enabling efficient query-time retrieval.

Our method gives up some preprocessing time for retrieval focus and inference speed compared to dense and hybrid baselines.

G. Summary

The suggested system is pretty cool because it uses a way of grouping things and focuses on the questions being asked to make searching easier. This system is good because it can handle more different sets of data even though we only tested it with a medium sized set of information. We think the system will work well with big sets of data and we will try that out in the future. The suggested system uses this grouping and question-focused searching to make things easier.

IX. Limitations

While the proposed system demonstrates improved grounding and practical applicability, several limitations remain.

A. Dependence on Question Generation

The quality of the questions that are created is very important for how the system works. If the questions are too general or not complete it can affect how well the system retrieves information, which can mean that it does not cover all of the knowledge.

B. Noise in Extracted Content

When the system is used with photos and PDFs it uses special tools to read the text, which can sometimes add mistakes because the quality of the input is not good or the design is too complicated. These mistakes can then cause problems in the steps of generating and retrieving information.

C. Evaluation Scale

The experiments were done using a sized set of data that is specific to a certain area. The results, about how the system can be scaled up are not definitely proven, but rather suggested and the test does not show what would happen in a real large-scale situation although it is enough to show how the system works.

D. Clustering Sensitivity

The way we group things together and the choices we make are really important for prototype-based indexing. If we do not group things together properly it can affect how well we can find what we are looking for. This is because some questions may be more common than others or some groups may overlap in what they mean.

E. Preprocessing Overhead

Our system needs to do some work before it can start helping us find things. This extra work includes grouping things and coming up with questions. This helps make it faster when we are actually looking for something. If the information we are searching through changes a lot this extra work can become a big problem.

F. Language Scope

Now our system only works with English. It does not work with languages or help with finding things in multiple languages at the same time.

G. Evaluation Constraints

We mainly use computers to evaluate how well our system is working. While this is helpful for looking at a lot of information it may not always match how people really think about the quality of the answers they get.

H. Summary

These limitations show us that we have to balance how much we can evaluate how work it takes to get ready and how well we can find what we are looking for. They also give us ideas for how we can make our system better, in the future.

X. Conclusion and Future Work

We made a tool to help answer questions better in this study. This tool is an improvement on something called QuIM-RAG. We want to use it in the world where people need to know things about specific topics. Our method uses kinds of information from the internet like tables, pictures with text and documents. We put all this information into one system that focuses on answering questions. This is different, from systems that just looked at pieces of text to find answers.

The method helps find answers in a step-by-step way. It makes the search smaller. Still matches what the user is asking. It does this by making example questions and grouping them together based on what they're about. The test results show that this approach works well in terms of understanding. It also does a job of being accurate and reliable. This means it is less likely to give answers that are not supported by facts. Some other methods, like getting information about the question and re-checking the answers can also make the process more reliable. These extra steps help without changing how the answers are found. The approach is not as good at recalling all the information, as some methods. It is still competitive and has some advantages. The method provides results in certain areas. It does this by being more accurate and reliable. The suggested method has benefits. It helps users get the information they need. The method achieves its goals effectively.

The study shows that the tool helped to make a pipeline to generate text and it works well with kinds of data from the web. Using prototypes, to index things can make it faster to find what you are looking for by narrowing down the options. This is important because it shows that we need to think about the data and the questions we are trying to answer when we design RAG systems that we will use in the world with real RAG systems.

A. Future Work

Several directions can further enhance the proposed framework:

- **Multilingual Extension:** We can expand the system to support languages and enable cross-lingual and multilingual retrieval.
- **Adaptive Re-Clustering:** The prototypes can be modified dynamically in response to changes in the domain and as the corpus evolves over time.
- **Robust Information Extraction:** We can improve OCR and document processing to handle inputs and complicated layouts.
- **Human-Centered Evaluation:** Human assessments can be included to evaluate the reliability and utility of the system.

This work shows that adding data collecting using various tools and retrieval processes to question-centric retrieval frameworks can make RAG systems more dependable and practical. The RAG systems become better when we use tools to collect more data and get the right information. This is because the RAG systems can give us accurate answers when they have more useful data to work with. The RAG systems are really helpful when they have data collecting and retrieval processes.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.

[2] S. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation with Inverted Question Matching," IEEE Access, 2024.

[3] T. Zhang et al., "BERTScore: Evaluating Text Generation with BERT," in Proc. ICLR, 2019.

[4] R. Shaham et al., "RAGAS: Automated Evaluation of Retrieval-Augmented Generation," arXiv:2309.15217, 2023.

[5] A. Maynez et al., "On Faithfulness and Factuality in Abstractive Summarization," ACL, 2020.

[6] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM TOIS, 2023.

[7] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, 2023.

[8] G. Mialon et al., "Augmented Language Models: A Survey," arXiv:2302.07842, 2023.

[9] A. Lazaridou et al., "Internet-Augmented Language Models through Few-Shot Prompting," arXiv:2203.05115, 2022.

[10] T. M. Breuel, "High Performance Text Recognition Using a Hybrid Convolutional LSTM Implementation," in Proc. ICDAR, 2017.

[11] S. Gupta and C. Manning, "Improved Neural Models for Web Table Understanding," in Proc. WWW, 2016.

[12] J. Lin, R. Nogueira, and A. Yates, "Pretrained Transformers for Text Ranking," Foundations and Trends IR, 2020.

[13] O. Khattab and M. Zaharia, "ColBERT: Efficient Passage Search via Late Interaction," SIGIR, 2020.

[14] L. Huang et al., "A Survey on Hallucination in Large Language Models," ACM TOIS, 2024.

[15] S. Wang et al., "A Survey on Evaluation of Large Language Models," ACM TIST, 2024.

[16] N. Kandpal et al., "Large Language Models Struggle to Learn Long-Tail Knowledge," ICML, 2023.

[17] T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.

[18] A. Vaswani et al., "Attention is All You Need," NeurIPS, 2017.

[19] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers," NAACL, 2019.

[20] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023.

[21] M. Klesel and H. F. Wittmann, "Retrieval-Augmented Generation in Enterprise Systems," BISE, 2025.

[22] Lins Rodrigues Cruz and João Luís, "Reinforcement Learning With Supervised Alignment for Grounded Truth," Stevens Institute of Technology ProQuest Dissertations & Theses, 2025.